

Chapter 5C: CSP Search (Part 2)

AMCS2104 Fundamentals of Artificial Intelligence

Content

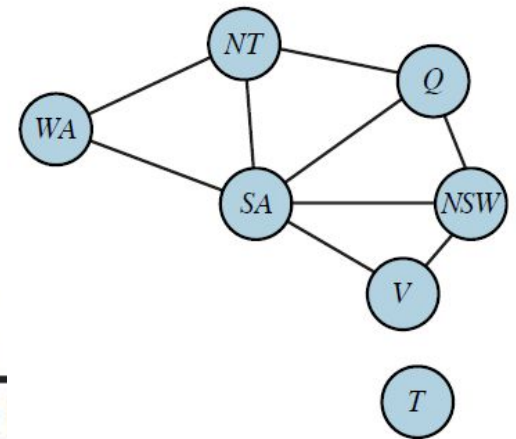
- Forward checking
- Forward checking with MRV Heuristic
- Maintaining Arc Consistency Algorithm
- Backjumping
- Conflict-Directed Backjumping
- Conflict Set
- Constraint Learning
- Local Search for CSPs

Interleaving Search and Inference

- Inference can be done before search, as a pre-processing step.
- Inference can be carried out in the course of a search too, in fact, more powerful.
- Inferences can be carried out after every assignment.
- To reduce the domains on the neighboring variables.
- **Forward checking** is a form of such inference:
 - Whenever a variable X is assigned, it establishes **arc consistency** for it.
 - For each unassigned variable Y connected to X by a constraint, delete from Y's domain any value that is inconsistent with the value chosen for X.

Forward Checking

- After the assignment of $\{WA = \text{Red}\}$, the domains of NT and SA are reduced.
- Then, after the assignment of $\{WA = \text{Red}, Q = \text{Green}\}$, the domains of NT and SA are reduced to only a single value.
- By propagating information from WA and Q, the branching on these two variables have been eliminated.



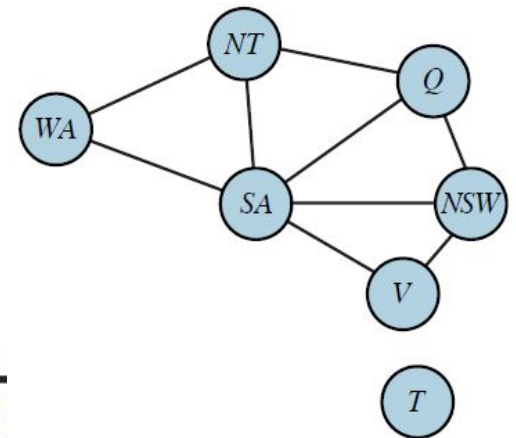
	WA	NT	Q	NSW	V	SA
Initial domains	R G B	R G B	R G B	R G B	R G B	R G

Forward Checking

- After the assignment of $\{V = \text{Blue}\}$, the domain of SA becomes empty.
- Hence, forward checking has detected that the following partial assignment is inconsistent with the constraints of the problem:

$\{WA = \text{Red}, Q = \text{Green}, V = \text{Blue}\}$

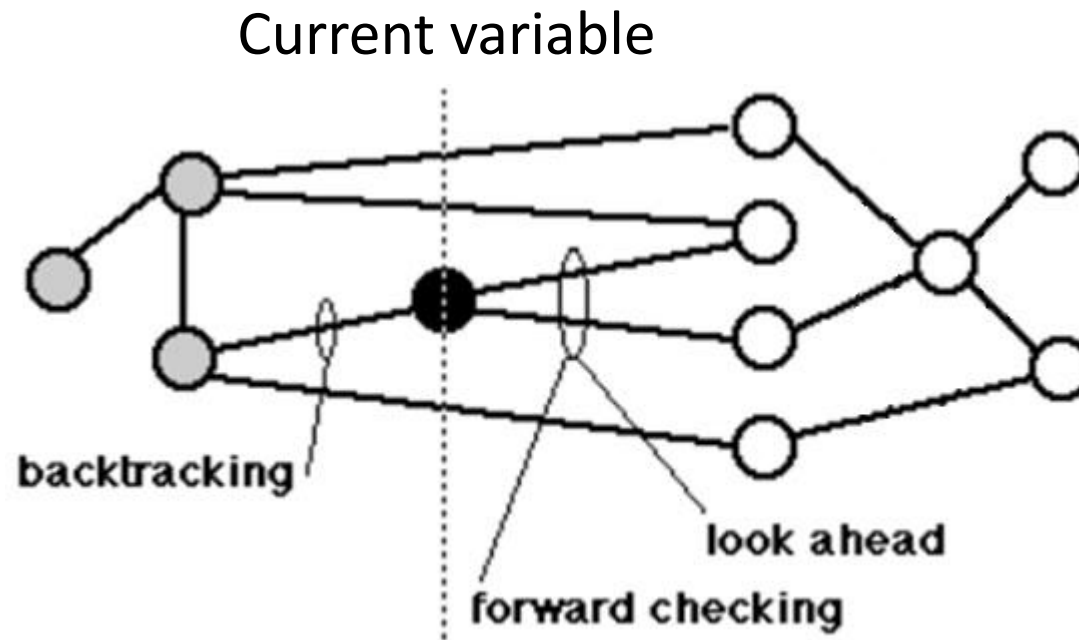
- The algorithm will therefore backtrack immediately.



	WA	NT	Q	NSW	V	SA	
Initial domains	R G B	R G B	R G B	R G B	R G B	R G	
After $WA=\text{red}$	Ⓐ	G B	R G B	R G B	R G B	G B	R G B
After $Q=\text{green}$	Ⓐ	B	Ⓔ	R B	R G B	B	R G B
After $V=\text{blue}$	Ⓐ	B	Ⓔ	R	Ⓑ		R G B

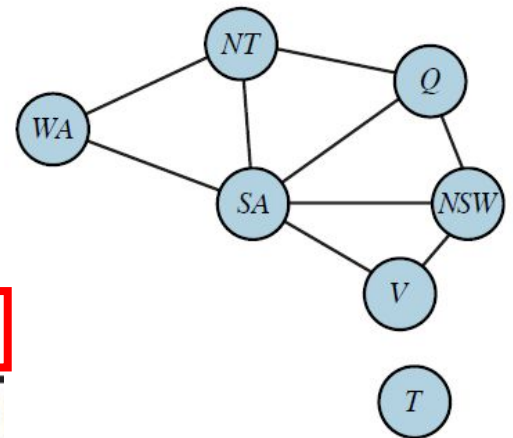
Backtracking Search vs. Forward Checking

□ **Question:** What is the difference between backtracking search and forward checking?



Forward Checking with MRV Heuristics

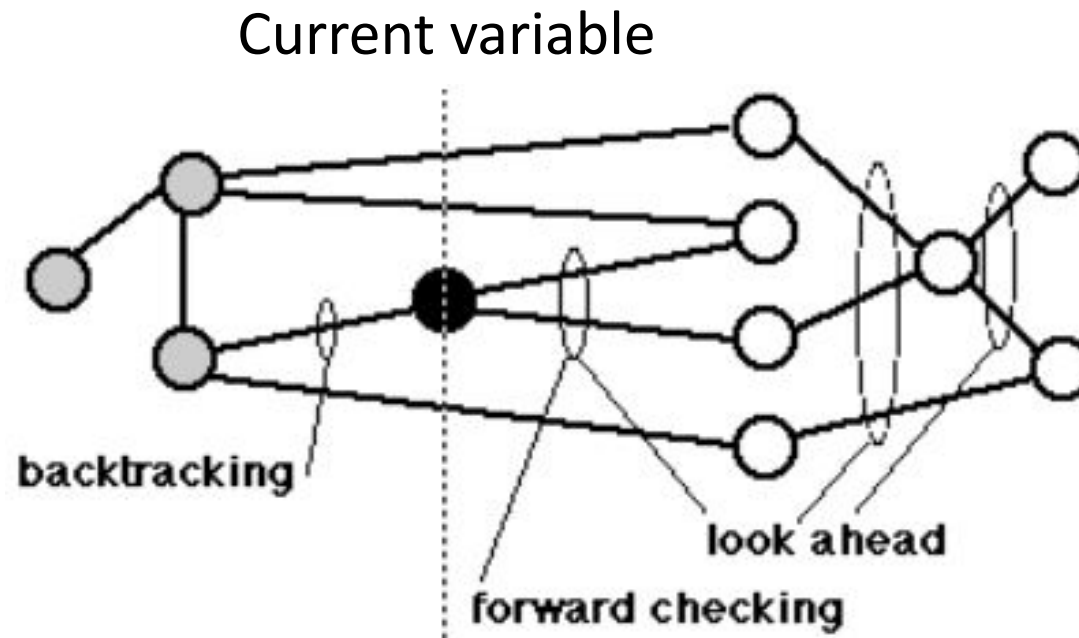
- A combination of forward checking and the MRV heuristic is more effective.
- Example
 - The assignment of $\{WA = \text{Red}\}$ reduces the domains of NT and SA the most.
 - They should be handled next before the other variables.



	WA	NT	Q	NSW	V	SA	
Initial domains	R G B	R G B	R G B	R G B	R G B	R G	
After $WA = \text{red}$	Ⓡ	G B	R G B	R G B	R G B	G B	R G B

Limitation of Forward Checking

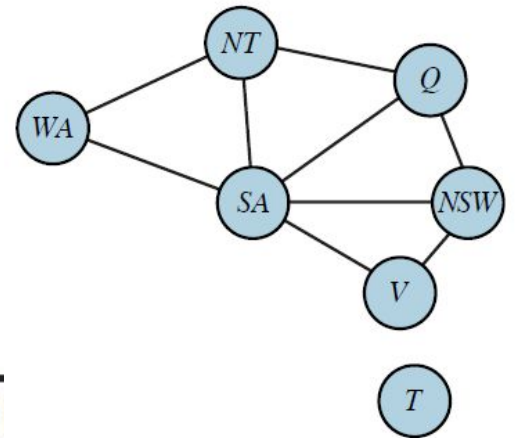
- Forward checking only makes the current variable arc-consistent.
- It doesn't look ahead enough and make all the other variables arc-consistent.



Limitation of Forward Checking

□ Example

- When $\{WA = \text{Red}, Q = \text{Green}\}$, NT and SA are forced to be $\{\text{Blue}\}$.
- It fails to notice an inconsistency, in which both NT and SA are adjacent and hence they cannot hold the same value.



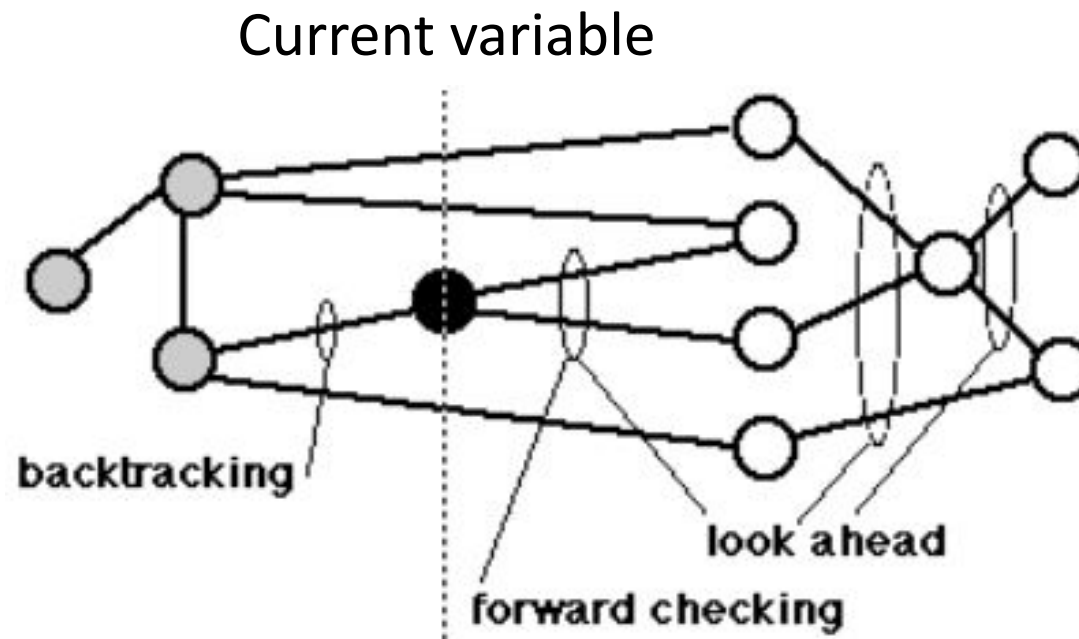
	WA	NT	Q	NSW	V	SA	
Initial domains	R G B	R G B	R G B	R G B	R G B	R G	
After $WA=\text{red}$	Ⓡ	G B	R G B	R G B	R G B	G B	R G B
After $Q=\text{green}$	Ⓡ	B	Ⓢ	R B	R G B	B	R G B

Maintaining Arc Consistency Algorithm

- The **Maintaining Arc Consistency (MAC)** algorithm can detect the inconsistency failed to be discovered by forward checking.
- MAC achieves this by **recursively propagating constraints when changes are made to the domains of variables**.
- Example
 - After a variable X_i is assigned a value...
 - Instead of a queue of all arcs in the CSP, MAC starts looking at the arcs (X_j, X_i) for all X_j that are unassigned variables that are neighbors of X_i ...
 - From here onwards, constraint propagation happens in the same way...

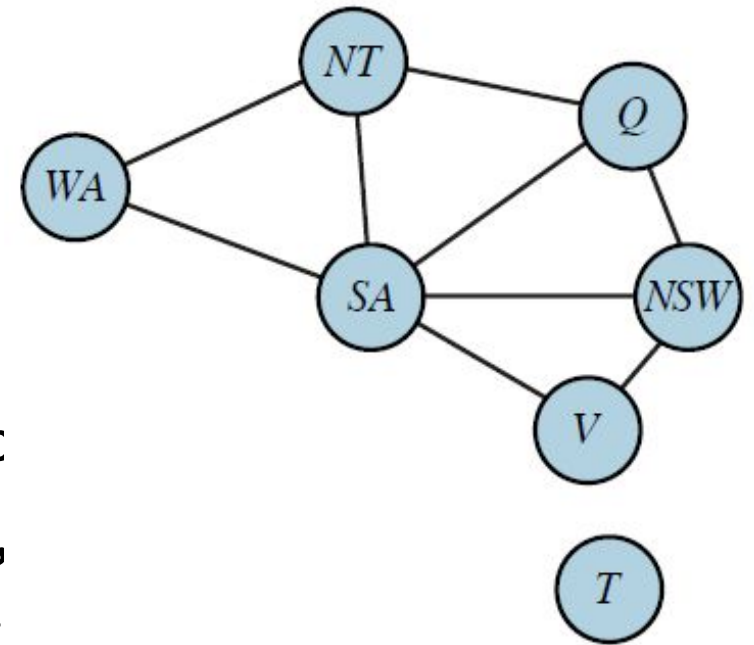
Maintaining Arc Consistency Algorithm

- MAC achieves this by **recursively propagating constraints when changes are made to the domains of variables.**



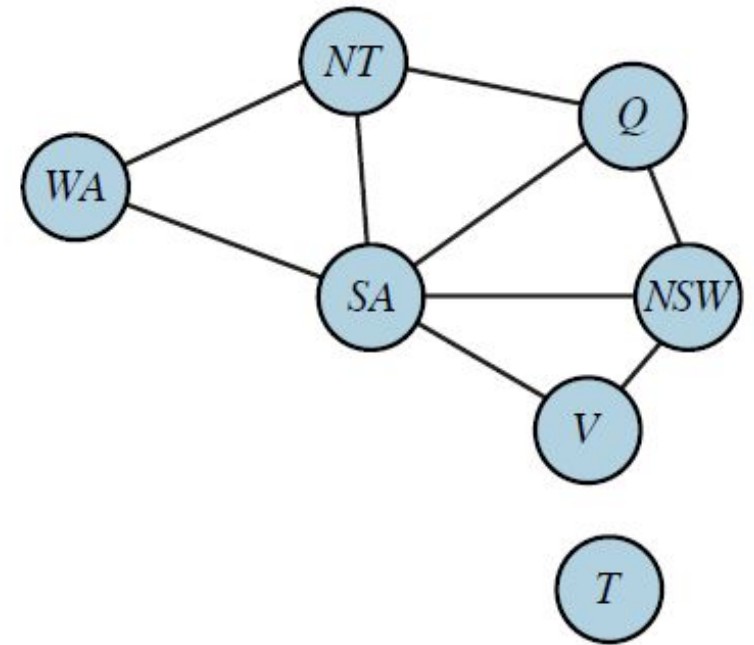
Intelligent Backtracking

- When the backtracking algorithm notices a branch of the search fails, it backs up to the preceding variable and try a different value for
- This is called **chronological backtracking** as the most recent decision point is revisited.
- Example: Map Coloring Problem
 - Suppose the assignment follows this ordering {Q, V, T, SA, WA, NT} and partial assignment has been generated {Q = Red, NSW = Green, V = Blue,
 - All the assignment of SA next violates a constraint
 - So, should backtracking search back up to T and try a new color?



Backjumping

- **Backjumping** is a more intelligent approach that **backtracks to the most recent assignment in a conflict set**, a set of assignments that conflict with some values for the current variable.
- Example
 - $\{Q = \text{Red}, \text{NSW} = \text{Green}, V = \text{Blue}, T = \text{Red}\}$ is the partial assignment and $\{Q = \text{Red}, \text{NSW} = \text{Green}, V = \text{Blue}\}$ is the conflict set for SA.
 - In this case, backjumping would jump over T and try a new value for V.



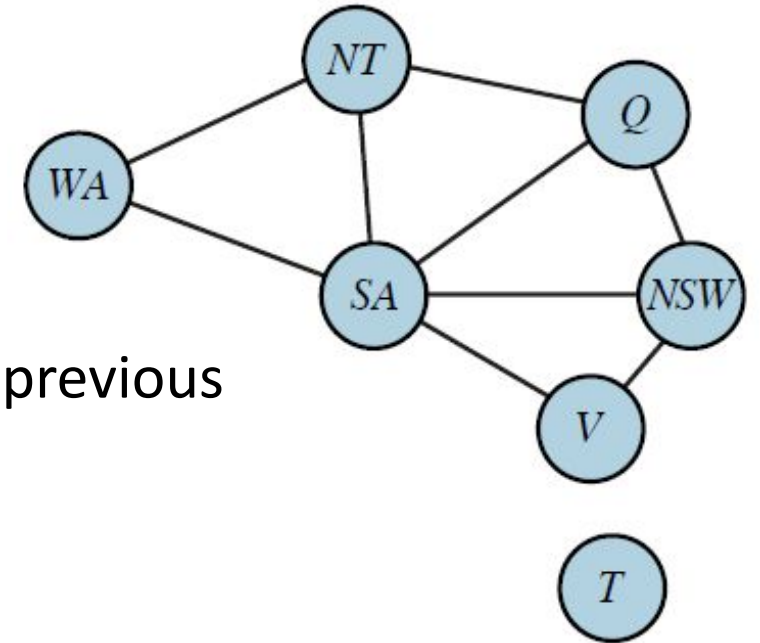
Backjumping vs. Forward Checking

- This method is easily implemented by a modification to backtracking algorithm:
 - Accumulates the **conflict set** while checking for a legal value to assign.
 - If no legal value is found, the algorithm should return the most recent element of the conflict set along with the failure indicator.
- Backjumping occurs only when every value in a domain is in conflict with the current assignment but forward checking detects this event and prevents the search from ever reaching such a node.
- Hence, a simple backjumping is redundant in a forward-checking search.

Backjumping

□ Example

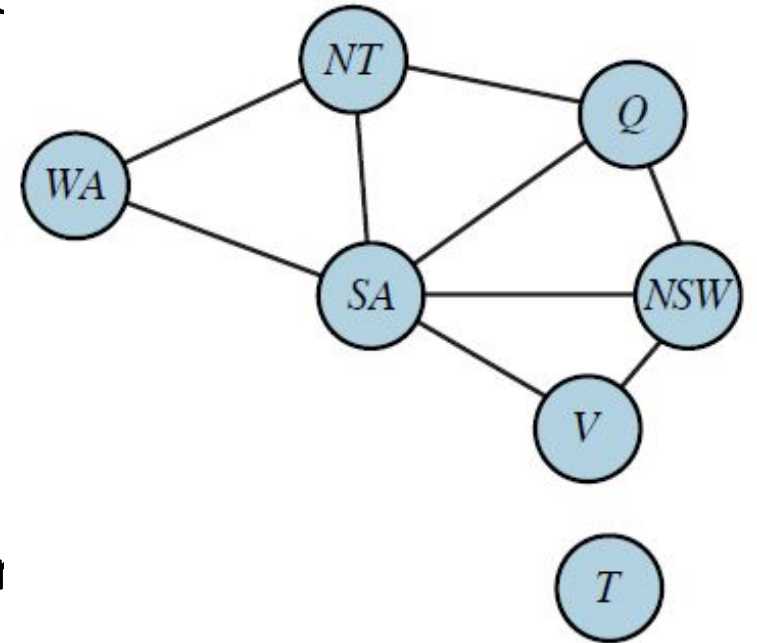
- Provided $\{WA = \text{Red}, NSW = \text{Red}\}$.
- Suppose the next assignment is $\{T = \text{Red}\}$ followed by the assignment for NT, Q, V and SA.
- NT, Q and V do not have any conflict set with the previous assignment except for SA.
- So, backjumping will try a new value with V.



□ **Question:** Does this solution work? If not, where to backtrack?

Conflict-Directed Backjumping

- It is obvious that the 4 variables NT, Q, V and SA failed because of a set of preceding partial assignments.
- In this case, the set is $\{WA = \text{Red}, NSW = \text{Red}\}$.
- They caused any subsequent variables to have no consistent solution.
- Thus, the algorithm should backtrack to NSW.
- A backjumping algorithm that uses **conflict sets** defined is called **conflict-directed backjumping**.
- When a contradiction is reached, backjumping can tell us how far to back up, so the search can fix the real problem.



Conflict-Directed Backjumping: Conflict Set

□ Example

- Suppose {SA} fails, and its conflict set is {WA, NT, Q}.
- The search backjumps to Q, and Q absorbs the conflict set from SA, which is {NT, NSW}, and the new conflict set becomes {WA, NT, NSW}.
- But there is still no solution found, and hence it backtracks to NT, the most recent of these, where NT absorbs {WA, NT, NSW} into its own conflict set {WA}, giving {WA, NSW}.
- Now, the search can look into the real problem, which is the assignment of {NSW = Red}.

Constraint Learning

- Once we reach a contradiction, we would like to avoid running into the same problem again.
- When the search arrives at a contradiction, we know that some subset of the conflict set is responsible for the problem.
- **Constraint learning** is the idea of finding a minimum set of variables from the conflict set that causes the problem.
- This set of variables, along with their corresponding values, is called a no-good.
- We then record the no-good.

Constraint Learning

- Consider the state {WA = red, NT = green, Q = blue} in the bottom row.
- Forward checking can tell us this state is a no-good because there is no valid assignment to SA.

	WA	NT	Q	NSW	V	SA	T
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After <i>WA=red</i>	Ⓐ	G B	R G B	R G B	R G B	G B	R G B
After <i>Q=green</i>	Ⓐ	B	Ⓔ	R B	R G B	B	R G B
After <i>V=blue</i>	Ⓐ	B	Ⓔ	R	Ⓑ		R G B

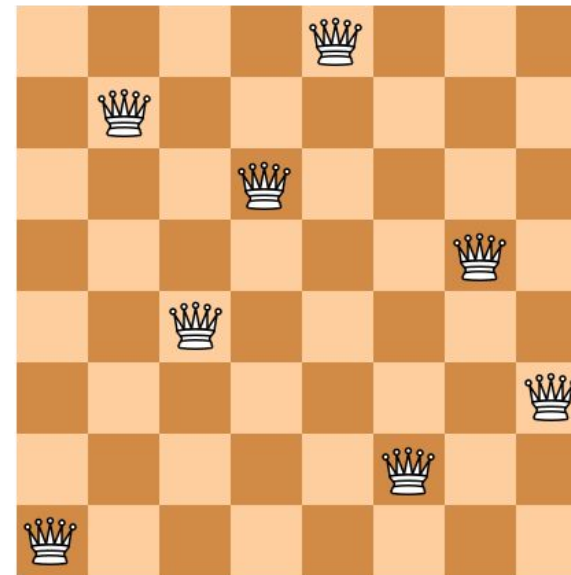
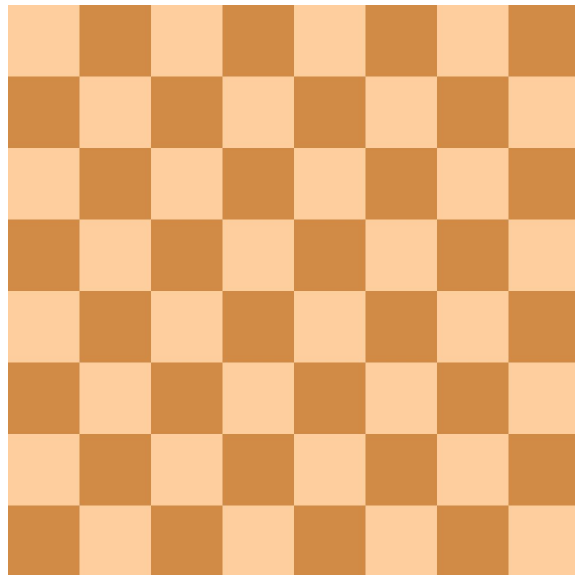
Constraint Learning

- In this particular case, recording the no-good would not help, because once we prune this branch from the search tree, we will never encounter this combination again.
- But suppose that the search tree (in the previous slide) were actually part of a larger search tree that started by first assigning values for V and T.
- Then it would be worthwhile to record {WA = red, NT = green, Q = blue} as a no-good because we are going to run into the same problem again for each possible set of assignments to V and T.

Local Search for CSPs

- Unlike backtracking search algorithm uses partial state formulation, **local search algorithms** use a **complete-state formulation**
 - Each state assigns a value to every variable.
 - Then, the search changes the value of one variable at a time.

Incremental
formulation



Complete-state
formulation

Local Search for CSPs

- With a complete-state formulation, the initial guess violates several constraints.
- The point of **local search** is to **eliminate the violated constraints**.
- When choosing a new value for a variable, the most obvious heuristic is to select the value that results in the **minimum number of conflicts with other variables**.
- Such heuristic is called the **min-conflicts heuristic**.
- All local search techniques studied earlier are candidates for application to CSPs.

Local Search for CSPs

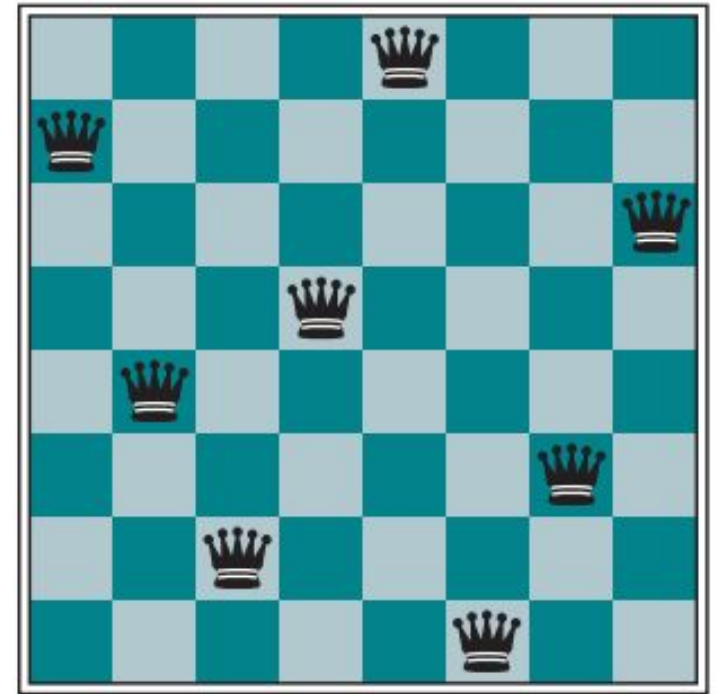
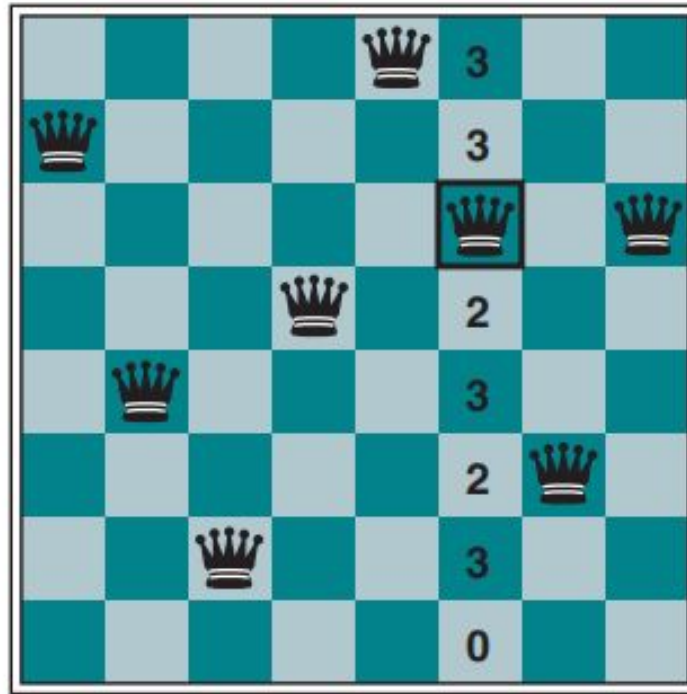
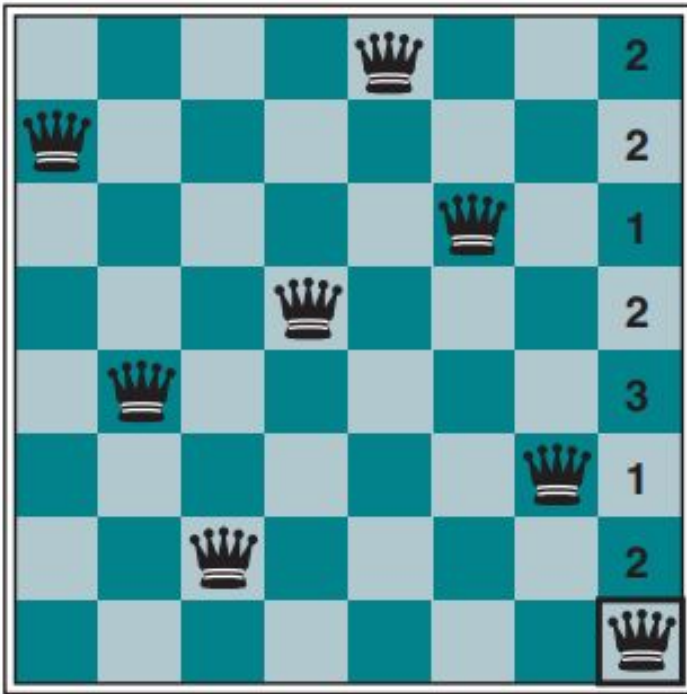
- The MIN-CONFLICTS algorithm for solving CSPs by local search
- The CONFLICTS function counts the number of constraints violation

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure  
  inputs: csp, a constraint satisfaction problem  
           max_steps, the number of steps allowed before giving up  
  
  current  $\leftarrow$  an initial complete assignment for csp  
  for i = 1 to max_steps do  
    if current is a solution for csp then return current  
    var  $\leftarrow$  a randomly chosen conflicted variable from csp.VARIABLES  
    value  $\leftarrow$  the value v for var that minimizes CONFLICTS(csp, var, v, current)  
    set var = value in current  
  return failure
```

Local Search for CSPs

□ Example

□ A two-step solution using min-conflicts for an 8-queens problem.



Local Search for CSPs

- **Min-conflicts heuristic** is surprisingly effective for many CSPs.
- It solves even the **million-queens problem** in an average of 50 steps.
- Issue
 - CSP's landscape (under min-conflicts heuristic) usually has a series of plateau.
 - There might be millions of variable assignments that are only one conflict away from a solution.
- Solution
 - Plateau search (sideways moves) can help local search find its way off this plateau.
 - Simulated annealing can also be used to escape from plateau.

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 5: Constraint Satisfaction Problems (page 178 to page 183)

The End